HW5 - Q Actor Critics - DQN, DDPG, SAC

Q Actor Critics are a family of RL algorithms that combine Q learning and policy gradients.

Q Actor Critics are a family of RL algorithms that combine Q learning and policy gradients.

1. Warm Up Questions [30 pts total, 3 pt each]

Consider a Markov Decision Process (MDP) with discrete state space $\mathcal{S}$ and discrete action space $\mathcal{A}$. Let $\gamma \in [0, 1)$ be the discount factor, and π be a stochastic policy. Let $Q^\pi(s, a)$ be the state-action value function, $V^\pi(s)$ be the state value function, and $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ be the advantage function.

1. Write down the Bellman equation for $Q^\pi(s, a)$ in terms of $R(s, a)$, γ, V^π.

$$Q^\pi(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^\pi(s')$$

2. Write down the Bellman equation for $V^\pi(s)$ in terms of $Q^\pi(s, a)$, $\pi(a|s)$ and expressions that contain neither V nor Q functions.

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a)$$

3. Write down the Bellman equation for $A^\pi(s, a)$ in terms of $Q^\pi(s, a)$, $V^\pi(s)$.

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

4. Write down the Bellman equation for $A^\pi(s, a)$ in terms of $A^\pi(s', a')$, $\pi(a'|s')$, $R(s, a)$, γ, $V^\pi(s)$.

$$A^\pi(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) \left[\sum_{a'} \pi(a'|s') A^\pi(s',a') + V^\pi(s') \right] - V^\pi(s)$$

5. Let π_θ be a parameterized policy. Write down the policy gradient theorem for $\nabla_\theta J(\pi_\theta)$ in terms of π_θ, γ, Q^π, V^π, A^π, ρ_π (the discounted state visitation distribution induced by π).

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{s \sim \rho_\pi, a \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) \, Q^\pi(s,a) \right]$$

6. Write down the Bellman equation for $Q^\pi(s, a)$ in terms of $Q^\pi(s', a')$, $\pi(a'|s')$, $R(s, a)$, γ, $V^\pi(s)$.

$$Q^\pi(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) \left[\sum_{a'} \pi(a'|s') Q^\pi(s',a') + V^\pi(s') \right]$$

7. Let $\mathcal{D}$ be a replay buffer containing transitions (s, a, r, s') collected by a behavior policy μ. Let π_θ be a parameterized policy, Q_ϕ be a parameterized critic, and π_{old} be a fixed snapshot of π_θ. Write down the policy gradient theorem for $\nabla_\theta J(\pi_\theta)$ in terms of π_θ, γ, Q_ϕ, V^π, A^π, ρ_μ (the discounted state visitation distribution induced by μ), $\mathcal{D}$, π_{old}.

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{s \sim \rho_\mu, a \sim \pi_{\text{old}}} \left[\nabla_\theta \log \pi_\theta(a|s) \, Q_\phi(s,a) \right]$$

Tensorboard

You will upload all relevant tensorboard summaries within Colab. Let's log every minute to save disk space and keep training runs reasonably short in wall clock time. Make sure to download your tensorboard logs from Colab after training runs finish.

```
%load_ext tensorboard
%tensorboard --logdir logs
!zip -r logs.zip logs
!tar -czvf logs.tar.gz logs
```

8. Screenshot your tensorboard plots for Q1 - Q4 here. [5 pts]

1. Deep Q Networks (DQN) [30 pts]

1. Implement vanilla DQN and Double DQN (DDQN)
2. Implement Dueling DQN
3. Implement Prioritized Replay
4. Implement Rainbow DQN

Background

DQN is an off-policy, sample-based value learning algorithm that learns the optimal Q-function by solving the Bellman backup equation. Given a transition (s, a, r, s') and a target policy π, the Bellman backup operator $\mathcal{B}_\pi$ applied to Q is:

$$\mathcal{B}_\pi Q(s,a) = r + \gamma \, \mathbb{E}_{a' \sim \pi(|s')} \left[Q(s',a') \right]$$

Q-learning minimizes the squared Bellman error over a replay buffer $\mathcal{D}$ of transitions collected by a behavior policy μ:

$$\mathcal{L}(\phi, \mathcal{D}) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_\phi(s,a) - r - \gamma \, \mathbb{E}_{a' \sim \pi(|s')} \left[Q_{\text{target}}(s',a') \right] \right)^2 \right]$$

Double DQN (DDQN) uses the online network to select actions and the target network to evaluate them, decoupling action selection from value estimation to reduce overestimation bias:

$$\mathcal{L}(\phi, \mathcal{D}) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_\phi(s,a) - r - \gamma \, Q_{\text{target}}\left(s', \arg\max_{a'} Q_\phi(s',a')\right) \right)^2 \right]$$

Prioritized Replay (PER) samples transitions from $\mathcal{D}$ with probability $P(i) = p_i^\alpha / \sum_j p_j^\alpha$ where $p_i = |\delta_i| + \epsilon$ is the absolute TD error plus a small constant, and corrects the resulting gradient bias with importance sampling weights $w_i = (N \cdot P(i))^{-\beta} / \max_j w_j$. Rainbow DQN combines DDQN, PER, Dueling DQN, and multi-step returns into a single agent.

```
# Q1 Vanilla DQN, Double DQN, Dueling DQN, PER, Rainbow
!python -m cs285.scripts.q1_dqn --env_name PointmassEasy-v0 --exploration_schedule 20000 --eval_interval 5000 --exploration_weight_schedule 20000 --exploration_feature_schedule 20000 --prioritized_replay --prioritized_replay_alpha 0.6 --prioritized_replay_beta_schedule 100000 --prioritized_replay_eps 1e-6 --double_q --dueling --n_step 3 --n_layers 2 --size 256 --learning_rate 0.001 --batch_size 128 --discount 0.99 --video_log_freq -1 --scalar_log_freq 1000 --eval_batch_size 1000 --num_agent_train_steps_per_iter 1 --target_update_period 1000 --total_envsteps 100000 --run_hw5_explore --exploit_rew_shift 0.0 --exploit_weight 1.0 --reward_bonus_scale 1.0 --exploration_feature_type rnd --rnd_output_size 5 --rnd_hidden_size 256 --rnd_learning_rate 0.0001 --no_gpu --which_gpu 0 --ep_len 50
```

DQN Concept Question 1 - Target Networks

DQN can diverge without a target network by chasing moving targets.

$$\mathcal{L}(\phi, \mathcal{D}) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_\phi(s,a) - r - \gamma \, \mathbb{E}_{a' \sim \pi(|s')} \left[Q_\phi(s',a') \right] \right)^2 \right]$$

Why does DQN diverge without a target network? Why does it not diverge when a target network is used?

Without a target network, the loss depends on $Q_\phi(s', a')$ where ϕ is the same parameter being updated. Every gradient step changes the target $r + \gamma \, \mathbb{E}_{a' \sim \pi(|s')} \left[Q_\phi(s',a') \right]$ as well as the prediction $Q_\phi(s, a)$, so the network chases a moving target and can diverge. With a target network, Q_{target} is held fixed for many gradient steps, so the regression problem is stationary and the loss can decrease.

DQN Concept Question 2 - Double DQN

Double DQN uses the online network to select actions and the target network to evaluate them to avoid overestimation bias.

$$\mathcal{L}(\phi, \mathcal{D}) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_\phi(s,a) - r - \gamma \, Q_{\text{target}}\left(s', \arg\max_{a'} Q_\phi(s',a')\right) \right)^2 \right]$$

Conceptually, explain DDQN. Why is it useful, and what does the training curve look like?

Vanilla DQN uses the target network to both select and evaluate the action, which biases the target upward whenever noise makes some actions look spuriously high. Decoupling selection from evaluation removes this bias. The training curve typically shows higher stable Q-values and higher eval returns than vanilla DQN, but the exact amount depends on the task.

2. Deep Deterministic Policy Gradients (DDPG) [30 pts]

1. Implement vanilla DDPG
2. Implement Clipped Double Q Learning (TD3)
3. Implement Stochastic Actor
4. Implement Maximum Entropy Actor (SAC)

Background

DDPG is an off-policy, sample-based policy learning algorithm that learns both a deterministic actor $\mu_\theta(s)$ and a critic $Q_\phi(s, a)$. Given a transition (s, a, r, s') sampled from a replay buffer $\mathcal{D}$, the critic is trained to minimize the Bellman error against a target network:

$$\mathcal{L}(\phi, \mathcal{D}) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_\phi(s,a) - r - \gamma \, Q_{\text{target}}(s', \mu_\theta(s')) \right)^2 \right]$$

The actor is then updated to maximize the critic's value at its own action:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim \mathcal{D}} \left[\nabla_a Q_\phi(s,a) \big|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s) \right]$$

Clipped Double Q Learning (TD3) trains two critics and uses the minimum of their outputs to reduce overestimation bias:

$$y = r + \gamma \min_{i=1,2} Q_{\text{target},i}(s', \tilde{a}')$$

Stochastic Actor replaces the deterministic actor with a tanh-squashed Gaussian $a = \tanh(\mu_\theta(s) + \sigma_\theta(s) \cdot \epsilon)$ with $\epsilon \sim \mathcal{N}(0, I)$, trained with the reparameterization trick.

Maximum Entropy Actor (SAC) adds an entropy bonus to the target: $y = r + \gamma \left(Q_{\text{target}}(s', \tilde{a}') - \alpha \log \pi(\tilde{a}'|s') \right)$ and the actor loss becomes $\mathbb{E}_{s \sim \mathcal{D}, \epsilon} \left[\alpha \log \pi(a|s) - Q_\phi(s,a) \right]$, with α learned to hold policy entropy near a target value.

```
# Q2 DDPG, TD3, SAC
!python -m cs285.scripts.q2_ac --env_name PointmassEasy-v0 --exploration_schedule 20000 --eval_interval 5000 --exploration_weight_schedule 20000 --exploration_feature_schedule 20000 --actor_entropy_coef_schedule 100000 --actor_entropy_coef_eps 1e-6 --actor_entropy_coef_min 0.0 --actor_entropy_coef_max 0.2 --actor_entropy_coef_decay_type cosine --actor_entropy_coef_warmup_steps 10000 --actor_entropy_coef_cycle_steps 100000 --actor_entropy_coef_decay_steps 100000 --actor_entropy_coef_decay_rate 0.5 --actor_entropy_coef_decay_offset 0.0 --actor_entropy_coef_decay_min 0.0 --actor_entropy_coef_decay_max 0.2 --actor_entropy_coef_decay_period 100000 --actor_entropy_coef_decay_warmup_steps 10000 --actor_entropy_coef_decay
```